AN OPEN-SOURCE MEMOIR



The Watch Party Story

From a few friends to Director's Cut

ORATORIAN

A story in ten chapters, with a prologue and epilogue

2024 — 2026

BEFORE PRESSING PLAY

About this account

This memoir is reconstructed in Oratorian's voice from the surviving project record: 1,008 unique commit messages, the 1.x changelog, both later release lines and their development summaries, the maintainer's Discord reflection, and the conversations surrounding the 3.0.0 release.

The private beginning comes from the maintainer's own recollection. The public chapters follow the releases and development history. Direct quotations are marked; scenes and conversations have not been invented. Earlier explanations are followed through their corrections, and unresolved reports remain unresolved.

The reading corpus includes 40 commits outside the present 2.x and 3.x branch histories, recovered through the 1.x version tags. A source trail at the back identifies the records behind the story.

MID-2024
A private project between friends

20 OCT 2025
The first public release

SPRING 2026
HLS lessons and the 2.0 rewrite

11 JUL 2026
Midnight Premiere becomes stable

AUG - SEP 2026
Director's Cut develops through betas

02 NOV 2026
Stable 3.0.0 reaches the community

THE RUNNING ORDER

Contents

The couch, eventually 4

A few friends, then a public release 5

The fix that needed another fix 7

Midnight Premiere, with additional subtitles 10

Everybody gets a pause button 13

The invisible character 15

Signed, sealed, readable 17

Director's Cut 20

Two servers, several subtitles, one film 23

One hundred percent of nothing 26

Beta4, then the couch 29

The next time it clicks 32

The record behind the story 34

PROLOGUE

The couch, eventually

Prologue | November 2, 2026

When 3.0.0 finally became a stable release, I had a suitable opening line for the announcement: "Took me long enough to get my booty on the couch and finally do something." It was a joke. It also left quite a lot out.

It left out the private project between friends, the public release, the shared stream that could not give everyone their own language, the seek fixes that required further seek fixes, and the beta releases people kept volunteering to test. It left out the detailed issues, the pull requests too large for a casual merge, the tests that passed while checking the wrong thing, and the afternoon when removing a dependency turned out to be better than updating it.

A changelog makes those events look reasonably orderly. Each version gets a heading. Underneath it, the work is divided into Added, Changed, Fixed and Security. The commit messages tell a more recognizable story. I thought I had found something, tested it, discovered another layer, corrected the explanation and tried again. Sometimes I said that fairly politely. Sometimes drift correction had failed us again and I wrote exactly that.

I kept the detail because I knew I would need it. Later, an old symptom would come back wearing a slightly different set of logs, and the explanation would be waiting. The record could remind me what I had already ruled out, what someone else had actually reported, and which apparently obvious idea had previously turned out to be nonsense.

CHAPTER 01

A few friends, then a public release

Mid-2024 to October 2025

Emby Watch Party started in mid-2024 as a small private project between friends. It was for me and a few others. There were three of us originally; the other two eventually moved on, and development continued largely as a solo project. The community came later. So did the name EWP, which people started using because apparently even a watch party needs a shorter invitation to itself.

The first public release went to GitHub on October 20, 2025. That is where the surviving public commit history begins, with version 1.0.0. By then there was already a recognizable watch party: a library, a video, synchronized controls and chat. The ambition was straightforward. Pick something from Emby, give people a party code, and watch it together. Getting all of those words to remain true at the same time would keep me occupied.

The first hour is a fairly honest introduction to the project. Release the app, explain remote access, fix a missing parenthesis, add a proxy. The proxy mattered because inviting friends to watch a film should not require exposing the Emby server itself to the internet. Watch Party could fetch and relay the playlists and video pieces while Emby stayed inside the local network. It was an early change with a practical consequence for anyone hosting this thing.

I also cleaned up the chat almost immediately. It had been announcing technical details about tracks, loaded video and stream readiness. Useful information for the developer, perhaps; less useful when several people are trying to talk. The chat gradually acquired some restraint. It

could tell people who had selected a video without reading the streaming machinery's diary aloud.

Then there were files that played without sound, and files that played the same two to four seconds repeatedly. Version 1.0.6 added a common browser-friendly audio and video profile. Everyone shared one transcode, which kept the server's workload manageable. That also meant accepting compromises: somebody with a lovely 4K display could still receive the same 1080p stream as everyone else. At this stage, finishing the film together had a certain advantage over individually admiring its resolution.

One audio fix removed the parameter selecting a particular language track. Twenty-one minutes later I added it back. Removing it helped one failure and broke files containing several languages. Subtitles had their own variation on this lesson. Text could be delivered separately and chosen by each viewer; image-based subtitles needed to be burned into the shared video. There was no single switch labelled make every media file behave. QuackMasterDan's dedicated testing and feedback helped expose the differences, and I put the thanks where people would actually see them, in the README and the interface. The public project was only a few days old. It already depended on somebody else trying the things I had missed.

CHAPTER 02

The fix that needed another fix

1.x and the spring of 2026

Synchronization became a recurring character. In October I added a check every five seconds to correct playback drift. Two days later I removed it. A late joiner was receiving an old stored position without the elapsed playing time added, and the correction mechanism was enthusiastically correcting the consequences. Fixing the initial position was more useful than repeatedly dragging everybody toward the wrong answer.

The app kept growing around that work. Skip Intro arrived, followed by themes, remembered library navigation and automatic next episodes. I gave the player gold borders and a red-curtain effect. Meanwhile, a VSCode auto-pull rebase lost several changes. Skip Intro retained its HTML and styling but lost its JavaScript, leaving the feature dressed for work without being able to do any. I reapplied it. The next-episode feature also needed to learn that every viewer should not independently select the next episode, and that the previous episode's final timestamp did not belong to its successor.

Other people helped make the surrounding application usable. MaaHeebTrackbee contributed the original Dockerfile and the optional login idea. Spexor suggested respecting library permissions. daniilkopylov requested a persistent party. wlowen and JeslynMcKenzie supplied testing, detailed reports and media information while quality selection and codec handling developed. The release names acquired a little personality too: Toasted Pretzel, then Synced Popcorn. A cheerful name did not grant immunity from synchronization problems.

THE DIFFERENCE THE LOG MADE

```
Requested codec:  h264
Observed output: -c:v:0 copy
```

Accepting a codec still allowed the source to be copied.

The historical HLS investigation turned on what Emby actually encoded, rather than what a request seemed to promise.

By March, heartbeat-based drift correction was back, with more cautious thresholds. In April I disabled it again after seek failures turned into loops. My commit message was fairly direct: 'sorry buddy, drift correction failed us again.' A player attempting to repair its position can make a bad stream much worse. The question had become why seeking itself was unreliable on certain files.

First I blamed peak bitrate. A file's average could look harmless while its difficult sections demanded much more. That explanation sounded useful enough to become a release. Direct requests to Emby's API then showed that the peak-bitrate field I was checking was never supplied. The check read nothing and achieved nothing. I described version 1.6.1 as 'chasing a ghost.' Next I tried explicitly requesting H.264. That produced another release and another correction.

The important detail was where Emby made its decision. Declaring that a client accepts H.264 does not necessarily tell Emby to re-encode existing H.264 video. It could copy the original video into streaming segments, retaining the source's irregularly spaced keyframes. Those frames are the useful starting points for decoding. If the playlist suggests tidy segment boundaries while the actual video does not line up with them, a viewer's perfectly reasonable seek can land somewhere unhelpful, fail, or return to zero.

Version 1.6.3 finally addressed the correct layer with EnableAutoStreamCopy=false on the streaming request. Emby's own ffmpeg logs showed real encoding instead of copying. The earlier instructions had influenced recommendations or described acceptable codecs; the streaming endpoint still made its own choice. I documented that distinction, along with measured hardware tests rather than my earlier estimates. The resulting quirks document grew out of these corrections. So did much of the reason for 2.0.

tripleredadam then caught pause and seek no longer responding after a disconnect and return. cimspin supplied precise subtitle reports which exposed contradictory streaming instructions and a missing Skip Intro button. The latter looked like a film without intro markers; Emby was actually refusing the request because I had used the wrong kind of credential. Detailed reports made these failures distinguishable, and therefore fixable.

CHAPTER 03

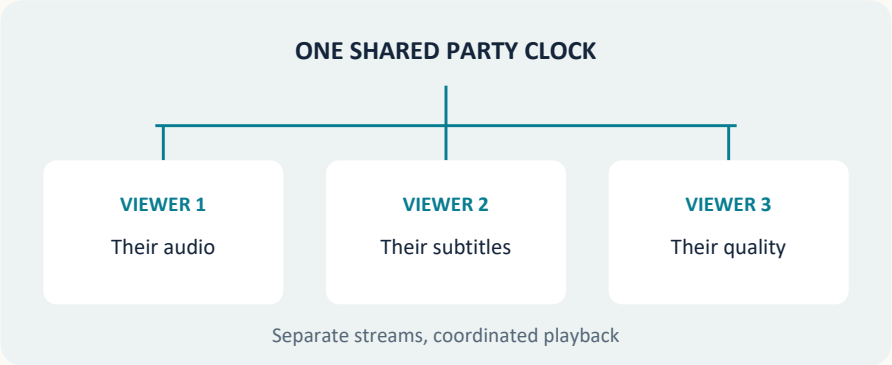
Midnight Premiere, with additional subtitles

The 2.0 rewrite and its closed betas

The 2.0 work began with an architectural cleanup in March and became a rewrite using FastAPI and Vue. The bigger change for viewers was independent streams. In 1.x, one shared transcode meant a shared choice of audio and video quality. In 2.0, people could make those choices for themselves. That was a useful improvement for a room whose browsers, connections and preferred languages were not identical.

Independent streams brought their own awkwardness. A person arriving halfway through could start a separate transcode whose frame boundaries did not visually match everybody else's. The late-join vote was a deliberate compromise: the people already watching could admit the newcomer and restart together. It gave everyone the same beginning. Changing a language or quality setting also needed care. An early coordinated pause-and-resume approach could leave the other viewers paused while the person changing settings carried on. I made those changes affect the requesting viewer's stream while the party continued.

NewBlade contributed eleven commits covering subtitles, seeking, reloads, ready checks and smaller-screen behavior. I brought them over with his authorship preserved. Integrating useful work still required testing how it met the rest of the app, and subtitles provided an unnecessarily elaborate demonstration. The chat began filling with reports that a viewer was seeking during ordinary playback. I blamed subtitle handling, changed the track management, disabled parts of HLS.js, then reverted the unified subtitle dropdown.



A simplified view of the per-viewer model introduced for 2.0.

The culprit in that episode was accumulated socket listeners after development reloads. Reload six times, keep six handlers, and one event gets six opportunities to announce itself. I fixed the stacking and restored the dropdown. Some of NewBlade's code remained as contributed, some evolved and some was superseded; the credit stayed. I also recorded the actual cause so a future visit to the problem would not begin with me confidently blaming subtitles again.

A production beta then produced another seek cascade, this time without development reloads. I briefly re-enabled forced transcoding as a defensive measure. The loop survived. A seek broadcast was returning to the person who made it, and the browser was being told to seek to the position it already occupied. A queued browser event escaped the synchronization guard and became another supposedly intentional seek. Around it went. Avoiding the redundant assignment stopped that loop. The same symptom had managed to collect several different causes.

The small features were capable of similar persistence. The emoji picker went through scrollbar changes, width changes, padding changes and a move outside a clipping container. A modest rectangle of smiley faces collected quite a commit history. Meanwhile Jeslyn's customized libraries exposed cropped collection art, missing metadata

and the browser tab still identifying the application as Vite App. It eventually received Tonight's Premiere, which was an improvement on continuing to advertise the scaffolding.

By May the first closed betas were running. The host model let one Emby-authenticated person provide access for a party, with sensible behavior when that host refreshed or left. Avatars arrived. So did bugs in votes, reconnects and stream changes. I kept writing detailed commit messages because an explanation that survives the fix is useful later, particularly when the original explanation turns out to need fixing too. Eighteen betas would sit between that first foundation and the stable release.

CHAPTER 04

Everybody gets a pause button

The road to July 2026

The later betas were shaped by the ways people actually organized and watched their media. Christian Gillinger's Refined Cyber mockup gave the interface its new visual direction. The controls became clearer, the current video acquired a LIVE badge, and the queued episode eventually acquired NEXT. Being able to see what binge mode intended to play was preferable to asking everybody to trust the word next.

xyxxyxxy supplied a steady stream of practical reports: enormous images downloaded for small library cards, a healthcheck that failed behind a URL prefix, libraries spread across multiple filesystem paths, and old requests arriving after somebody had already navigated back. Those were real setups exposing assumptions in mine. Resume prompts, playback progress on library cards, timestamp entry and binge watching made the experience more convenient. Discord contributors, including StealthyDruid, received thanks alongside the people reporting on GitHub and the Emby forums.

There were also quieter failures with larger consequences. A network connection could reconnect successfully while the viewer never rejoined the actual party room. The interface looked alive, but playback commands went nowhere. A library rescan could leave an empty view pinned in saved navigation, with an upstream error disguised as an empty folder. Both needed the app to recover properly. Asking somebody to refresh or restart a container might get that evening moving, but it also leaves the same trap waiting for the next person.

In July, a python-socketio security advisory prompted a wider audit. It confirmed twenty-seven issues. Among them was an admin API key included in stream URLs visible to viewers, even though the proxy already had the credentials it needed. Other findings involved reclaiming host access, session persistence, request limits and races in automatic episode advancement. I removed the exposed key, tightened the checks and worked through the functional consequences. The manual verification covered actual playback, subtitles, quality, avatars and refreshing in the middle of an episode countdown. The legacy 1.x line received its own security patch as 1.6.7.

Main moved to the rewritten application on July 9. Midnight Premiere became stable 2.0.0 on July 11. There was a migration guide because this upgrade changed authentication, configuration and stored data locations. Settings opened inside the party so visiting the admin panel did not destroy the player. The release had accumulated a great deal more than a replacement framework. It also carried the work of the people who had kept trying it and reporting precisely what happened.

Three days later, 2.0.1 returned to a basic expectation: 'if one pauses, everyone pauses.' Earlier hardening had restricted playback control to the host or selector. Shared control was the intended behavior, so I changed the server check to recognize every joined party member. That exposed browser-generated pauses and a recovery seek which the old restriction had accidentally concealed. The logs finally showed why pressing Play could immediately pause the room. I put the suppression around the synthetic operation where it belonged, checked the multi-user flow, and released the patch. Everybody could pause the film again. The player simply had to stop mistaking its own housekeeping for somebody pressing the button.

CHAPTER 05

The invisible character

A film stuck at 0:00 introduced the next chapter of the project.

Getting 2.0 into a stable release did not persuade Emby to stop being Emby. The rewrite had given viewers their own audio, subtitles and quality settings, and it had given me a considerably more interesting collection of ways for playback to fail. By this point my commit messages were becoming miniature investigations. Apparently I could write quite a lot about a video that had done absolutely nothing.

On August 1, dnordel contributed a fix for exactly that situation. Playback sat buffering at 0:00. The browser never requested the next playlist or the actual video segments, and Emby eventually decided the session was idle. From the viewer's side, there was very little to distinguish a broken stream from a server taking its time. The interesting part was a character nobody could see.

An HLS playlist is a small text document telling the player where to fetch the next pieces of video. Watch Party rewrote those addresses so the requests would pass through its proxy with a party token attached. Emby's playlists used Windows-style line endings, a carriage return followed by a newline. Our splitting left the carriage return attached to each address. We then helpfully added the token after it. The player read the result as a broken extra line, and the evening's entertainment remained 0:00.

dnordel's repair respected the incoming line endings and kept each address and token together. His changes also brought tests through the public HLS routes, covering the first playlist, the next playlist and the video segments. I merged the fix as PR #44 and released 2.0.2. One invisible character had managed to prevent the browser from reaching any visible frames. There was something almost economical about it.

The follow-up was less economical. I found a playlist request missing the timeout used by its neighboring requests. A slow Emby server could leave that work hanging until the operating system gave up. Looking at the new tests also exposed places where our substitutes were being more accommodating than the real application. A token check did not actually check membership, and a request could lose its upstream authentication without making the suite complain. I tightened those assumptions. Green tests needed to mean that the important parts had actually happened.

This is one reason I am glad detailed issues exist. A report that says where playback stopped, what the browser requested and what Emby eventually did gives a future investigation something solid to stand on. My own answer was apparently to go bezonkas in the commit messages. There are worse habits for a maintainer to develop, although anyone expecting a three-word fix message had picked the wrong repository.

dnordel's involvement grew far beyond that carriage return. His next work reached through application startup, party state, the Emby connection, browser behavior and testing. Some pieces were immediately useful on the stable line. Others changed the application's foundations. I was looking at a contribution large enough to alter what the next version of Watch Party could be.

I later said on Discord that his work revived the project and my passion for it. That was the honest description. The original collaborators had moved on, development had continued largely solo, and I had hit a dead end with 2.x. Here was someone bringing substantial improvements in functionality, code quality and the interface, all together. The stable release still needed maintenance. Alongside it, there was suddenly a much bigger conversation taking place in code.

CHAPTER 06

Signed, sealed, readable

The party looked healthy. The assumptions needed a closer look.

The security work around 2.1.0 forced me to be very precise about what each credential actually proved. That sounds like a sentence from a security document. In practice it meant finding out why someone could see a party, chat normally, appear in the member list and still receive absolutely no video. The application could present a convincing impression of success while its most important request was being refused.

The HLS routes had been relying on the token inside the stream address. Our documentation already described a session-cookie gate, but the code had missed it. A copied address was therefore carrying more authority than the documentation suggested. I added the gate, then checked the relationship between its two credentials. That second step mattered. A cookie for party A and a leaked stream token for party B could each pass their own check. Together they could still open the wrong party's stream.

The cookie and the stream token had to agree about which party they belonged to. Once that was enforced, another old shortcut became visible. Joining over HTTP bound the browser session, while the socket handled the live party connection. The frontend had treated failure of the HTTP step as something it could swallow and carry on. Stricter video authorization made that a particularly unhelpful courtesy. Chat worked, the room looked populated, and every video request failed. I added a retry and a banner that said plainly what had happened and offered a way to recover.

The cookie itself had a more serious surprise. Starlette signed its session data, which protected it against modification. The contents

remained readable. We had stored an Emby administrator access token inside that readable data. Anyone holding the cookie could decode the payload and recover a credential with authority over the Emby server. The word signed had been doing a remarkable amount of imaginary work.

I moved those administrator credentials into a server-side store. The browser received an opaque handle, and logging out revoked the stored credentials. The repair also removed the old credential fields when an administrator logged in again. This changed what a restart meant: remembered administrator logins disappeared with the process. That consequence belonged in the release notes alongside the security explanation. These were bugs we repaired during that development period; they are part of the history of getting the boundaries right.

There was a browser-shaped price to the stricter stream gate too. Cookies are shared across tabs, and our session cookie held one party identity. Open a different party in another tab and the first tab's video could stop. I made the tabs announce that handover and show which party had taken over. Then I changed the recovery button to say what clicking it would cost: Resume here, with the other party named as the one that would stop. My commit called it honestly a trade rather than a fix. Two tabs could otherwise spend the evening reclaiming the same cookie from each other.

While reviewing the rework, dnordel found another identity gap. The host's client identifier was broadcast to the party, and joining accepted an identifier supplied by the caller. Comparing those strings could therefore make a spectator look like the host. Even a correctly signed cookie could contain an identity the caller had chosen. I introduced a separate secret host grant, minted on the server and rotated when hosting changed. A name tag and proof of ownership were finally doing separate jobs.

I gave the earlier cookie and HLS changes a minor release number, 2.1.0, because people would notice the changed authorization and restart behavior. Calling something a patch would not make those consequences smaller. My commit messages spelled out the attacks, the reproduction steps and the limitations. If the explanation occasionally occupied more space than the fix, at least the next person investigating it would have something more useful than security improved.

CHAPTER 07

Director's Cut

dnordel's rework grew into the next major version.

PR #45 was too large a change to treat as an ordinary addition to the stable version. dnordel had worked on the machinery that decided how parties existed, how requests reached Emby, how browser messages were understood and how all of it could be exercised in tests. It changed Watch Party at its core. The work became the foundation of 3.0, on its own integration branch, while 2.x continued alongside it.

I still took useful, self-contained pieces across to the stable line. The iOS player could explain when autoplay was blocked, release its native stream when the component closed and fit the usable screen area properly. The Emby login dialog kept keyboard focus inside it and returned focus when it closed. Those were dnordel's fixes, selectively adapted for the existing application. Other pieces depended on his new architecture and test harness. Trying to pull everything over separately would have meant assembling another version of the rewrite by hand.

One important change concerned the messages traveling between browser and server. A party depends on both ends agreeing what join, seek, ready and stream-changed actually contain. An unexpected value at that boundary can become a viewer who never starts, a control that appears to work or a reconnect that restores only half the room. dnordel added typed contracts and validation, with generated browser definitions. The agreement became something the tools could check instead of an understanding maintained across several handwritten copies.

He also reorganized the party itself into explicit state and moved transitions through the party manager. That mattered when things overlapped: someone selected a video while another request was still

finishing, a viewer left during setup, or a browser returned with a fresh socket connection. The aim was visible in those ordinary actions. A departing viewer should not leave an orphaned stream running, and an old selection should not arrive late and replace the title the host had just chosen.

The application factory gave tests a way to create their own isolated application instances. The Emby gateway made upstream operations asynchronous and brought retry behavior into one place. A slow request to the media server should not hold the party's live controls hostage. These changes were substantial even when a viewer could describe their effect simply as the pause button responding when it was pressed.

The testing work reached beyond calling individual functions. dnordel built a fake Emby server and browser flows that went through live public interfaces. Two browsers could join, receive playable HLS and exercise synchronization. NewBlade's earlier reload, identity and ready-check work had already made the 2.0 architecture more usable; this next contribution gave us a much larger way to check the whole journey. Both contributions belong in the story. I inherited useful structure from people who had taken the time to understand the awkward parts.

The fake server also had to earn its credibility. Later checks found that its tidy playlists hid shapes real Emby used, and its responses could be too forgiving. That led to more faithful fixtures and tests that were deliberately checked against broken code. I found my own contribution to the same category of surprise: importing the application module could build the app and print a setup credential. Pytest's output capture had kept that quiet. As I wrote in the fix, that was luck, not a control.

By early August the branch needed a version that admitted what it was. I changed it to 3.0.0-dev and gave it the codename Director's Cut.

Midnight Premiere had established the cinema theme; this was a rebuilt version of the same film. The name suited the amount of work already sitting behind it. The next task was getting that work through real upgrades, different deployments and the community's browsers, where a technically respectable implementation would still have to manage the basic business of letting everyone watch together.

CHAPTER 08

Two servers, several subtitles, one film

August 2026

The HEVC report arrived with a fairly reasonable question: why was Emby re-encoding a video the browser could already play? miakkia had measured Direct Play when the source codec was preserved. My stream requests, meanwhile, ordered H.264 for everybody. That was a safe choice when I knew nothing about the viewer's decoder. The increasingly obvious problem was that I had never asked.

Browser names turned out to be a poor substitute for asking. Chrome needed hardware acceleration enabled. Firefox on Windows needed the appropriate Microsoft Store codec. A codec pack could contain everything necessary for a desktop player and still do nothing for the browser. I even corrected one of my own fixtures after blaming a machine's hardware when Chrome's acceleration setting was the actual difference. The same browser on the same computer could give a different answer between visits. So capability became something the page measured when it loaded.

Each viewer already had an independent stream, which gave the answer somewhere useful to go. A capable viewer could receive HEVC while somebody beside them in the participant list received H.264. Against the measured 8K source, copying three seconds of video took about fifty milliseconds of server work; encoding it at 1080p took roughly seventeen hundred and fifty. Those figures belonged to that test, rather than to every possible server, but the difference made the point quite comfortably. The work shipped as 2.1.2 and then moved into 3.0 so an upgrade would preserve it.

At the same time, dnordel was doing substantial work on the library. Filters, grouped search, an alphabet rail backed by the server's index, title details, seasons, episodes and playlists made the browser much more useful. Integrating his stacked pull requests required more care than pressing Merge. One conflict combined his correctly guarded session join with my codec field. Picking either complete side would have lost something: successful session binding on one side, HEVC negotiation on the other. I kept both, checked the reconnect paths, and explicitly recorded that assembling the stack did not mean I had finished reviewing it.

Actually using the title view supplied its own review. Related titles, extras and trailers could all remain expanded, pushing the page downward. A season could contribute twenty-four more rows. A sufficiently enthusiastic synopsis could send the controls off the screen before anyone touched them. We ended up with one bounded panel, two season and episode selectors, and controls beside Back, where an overview could no longer relocate them. The primary button eventually became Set up playback because that was what it opened. Calling a form Play had been rather optimistic.

Jellyfin then made the provider boundary earn its existence. dnordel built most of that implementation: normalized library data, authentication, playback negotiation, provider-owned streams and opaque resource identifiers that kept upstream URLs and credentials behind the server. His real-server journeys exercised official Jellyfin 10.10.7 and 10.11.11 images, including playback and transcode cleanup. The feature had clear limits, including HLS playback and supported Jellyfin versions. Having a second server family was already enough work without silently promising Live TV and everything else it could do.

The family resemblance still concealed some excellent traps. Sending null for a subtitle index allowed Jellyfin to choose one according to the account's preferences. Minus one was the explicit request for none. A

text subtitle could otherwise be burned into the picture while the frontend displayed its own copy. Turning on subtitle renditions in the manifest offered yet another route to an unwanted encore. I had to follow each delivery path instead of assuming the word subtitle meant one thing everywhere.

Quality settings were equally literal. A suffix parser gave several low-resolution choices the same fallback bitrate as Auto, while their resolution limits never reached Jellyfin. The bottom of the menu could cost more than the middle. Real-server testing then showed that the resolution-only tiers needed Jellyfin-specific bitrate floors as well. Emby had been choosing sensible values for itself. Carrying its behavior into an adapter required checking what the other end actually accepted, down to the setting Jellyfin explicitly refused. The provider abstraction made those differences possible to contain. It did not make them disappear.

CHAPTER 09

One hundred percent of nothing

The checks behind Director's Cut

One of the coverage gates reported one hundred percent while measuring zero changed Python lines. The report used one form of a filename and Git used another, so the two sets never met. Perfect coverage was apparently available to anyone willing to avoid counting the code. That finding rather neatly described the next part of 3.0: going through the reassuring results and establishing what each one had actually proved.

The container playback job could not reach the fake Emby it started because the fake listened on loopback while Docker connected through the bridge address. Another job downloaded actionlint using filenames that did not exist, then stopped before its contract and deployment checks could execute. The merge gate correctly failed that job, which mattered. Different green lights and red lights had different causes; flattening them into a general claim that CI was broken would have been as unhelpful as trusting all of them automatically.

The fake media server needed scrutiny too. It accepted unknown item identifiers, supplied too little variety and could not express several upstream failures. A TV library could become empty when filtered because the frontend and backend kept different maps of what belonged in it. An unavailable server could become No items found because an error had been swallowed into an empty result. These were practical consequences for viewers. Nobody looking at an empty grid should need to determine whether the library, the network or my interpretation of an exception had gone missing.

A captured-response corpus brought a more serious lesson. Its sanitizer preserved fields that could identify library content, while the leak check searched for markers the sanitizer already removed. I repaired the corpus and then corrected my own description of the exposure. The affected history had not reached an Oratorian branch; it was in the stack intended for integration. That history was rewritten and checked before landing. The contributor's fork and GitHub's retained pull-request reference were separate matters. Getting that distinction right was part of fixing the incident, even though it made the explanation longer.

The first library audit confirmed sixty-four findings. A second pass looked at the tests themselves and confirmed forty-seven more. I do not need to turn the rest of this chapter into a scoreboard. The useful detail is how we checked them: deliberately change the source in the way an assertion claimed to detect, then see whether the test failed. One end-of-video test calculated its expected result from the same constant it was supposed to protect. A thirty-minute safety buffer passed quite happily.

We applied sixty-four named mutations and made the tests catch every one. Some of the corrections exposed application bugs directly, including a dead-party flag following somebody into the next healthy room and a stream reporting the old clock after starting at a corrected position. The audit also caught inadequate proposed assertions. I kept those corrections in the record. Earlier commit messages had already become detailed enough to qualify as light reading with a dependency problem; at least the detail could explain what a future reviewer should distrust.

Operators supplied evidence our fixtures had missed. xux1217 reported that saving settings failed when config.json was mounted as a single file. The report named the failing call, the error number and the exact lines. The diagnosis was already done. Renaming over a mount point failed, so the recovery needed to preserve the existing inode. I chose

the narrow fallback, added tests for ordinary files and the mount-point fallback, and carried it onto both release lines. Credit for finding the fault belonged with xux1217, rather than disappearing behind the name on my commit.

The upgrade instructions needed the same discipline. Deployment examples came from one schema. Migration preflight read configuration without changing it and used the application's startup verdict. Retired provider variable names named their replacements instead of quietly falling back to localhost. Removing the old API routes meant documenting their successors and first rescuing behavior the new path had missed. Even a rejected configuration got an observable state: the container stayed available for diagnosis, printed the failing fields and answered readiness with 503. On an appliance with a log viewer, that explanation was considerably more useful than another restart.

The beta labels needed care, too. Beta2 was real development work, but it never had an image of its own; beta3 carried it. A guide that told somebody to run a new preflight from the old image could fail before it inspected their settings. Once we were documenting other people's upgrades, release notes had become part of the software's behavior.

CHAPTER 10

Beta4, then the couch

September to November 2026

On September 12 I wrote to the community about where the project had ended up. By beta4, Emby Watch Party had grown much bigger than I had anticipated. I needed time to decide when and how 3.0 should become the main release, so I said beta4 would be the last testing release for the time being.

I also said what dnordel's contribution had meant to me. His work had revived the project and my passion for it after I had reached a dead end with 2.x. The pull request was large because it changed the foundations, and those changes became 3.0. That belongs in the story alongside the review findings. His contributions added functionality and a better interface as well as structure. Calling everything my rewrite would have been a convenient way to lose the person who had helped it continue.

A pause in releases still leaves a project with people using it. The host-refresh and admin issues needed attention in October. A delayed status response could overwrite newer information from the socket; a successful login arriving late could undo a newer takeover. The admin panel could disappear after a recoverable loading failure and take the person's edits with it. I worked through those recovery paths and exercised them in Chromium and Firefox. Refreshing a page should not turn ownership into a race between whichever request happens to answer last.

Playback reports also continued. In August we had identified one reset mechanism: hls.js reattached after a media error, my listener loaded the source again, and the recovering client pulled the room back toward zero. That specific path was repaired. The later report from Ged

needed its own investigation, and the evidence did not establish the same trigger. I could document what the logs showed and the available workaround. I could not promote that into a claim that 3.0 had definitively fixed his incident. Detailed issues were valuable precisely because they left something concrete to return to.

Then November 2 acquired a substantial dependency population. I described the rhythm as Dependabot closing four and opening four, followed by a fairly economical WTF. Several updates needed more thought than accepting the generated lockfile. Linux regeneration had dropped Windows requirements and platform exclusions. I preserved colorama where Windows needed it and kept uvloop and librt out of platforms that should never receive them, checking more than the machine on which the lock happened to be generated. Supporting Windows had to survive the security update as well as the original installation.

Axios offered a particularly satisfying reduction in scope. The frontend already used native fetch and had no Axios consumers. Removing the unused dependency took twenty-five packages with it and resolved that chain without adopting compatibility changes for an API the application no longer called. The test tooling needed its own care: Vitest and coverage moved together to the complete patched release, rather than stopping at an intermediate version with a Windows bypass still outstanding. Certificates, HTTP clients and parsers all received attention. These changes had different exposure, and the release notes needed to say so.

The final preparation also caught up the documentation with the application people would install. The generic frontend starter guide became an actual development guide. Current migration requirements moved into one stable release entry while the beta notes remained available as history. The backend, frontend and multiuser playback checks passed before publication. On November 2, Director's Cut became stable 3.0.0, with latest following it. The older prerelease

pointers kept their own destinations. The 2.x line remained separately available for security maintenance, and 1.x retired.

The announcement could therefore afford its joke about how long it had taken me to get my booty on the couch and finally do something. The forum release and migration information went first, so the humor had instructions behind it. I also made the thanks explicit. StealthyDruid and xyxxyxxy had tested every beta release of both 2.x and 3.x, repeatedly returning with reports and feedback. A stable release at the end of that sequence carried a lot of other people's persistence. Their names belonged in it.

EPILOGUE

The next time it clicks

Epilogue

Near the end of the 3.0 preparation, another playback investigation sent me back to the old issues. The symptom was familiar: a movie reset, and other people were pulled back with it. Familiar was a starting point. It was not a diagnosis. The evidence for the previously repaired reset had involved repeated requests for the master playlist; this newer session needed its own explanation.

Then break_non_keyframes came up, and my brain caught up with the rest of the conversation. Wait a minute. I had seen this before. The old issues, the ffmpeg commands and the document with the rather accurate title Emby Quirks We Learned the Hard Way were still there. So were the corrections to my earlier confident answers.

That is why I am glad issues exist, and why detail matters. A report about a film that misbehaved in April can become useful again months later. A careful description survives the immediate frustration. It gives someone a way to recognize the shape of a problem without pretending that every similar symptom has the same cause.

Even then, I had to keep the limits of the evidence intact. A source configured for a long maximum keyframe interval supported the old line of investigation. It did not tell me the exact spacing at the troublesome moment or prove that the particular reset had been solved. Force Transcode was an established option for the media that needed controlled keyframes, with a real resource cost behind it. An option and a hunch still needed a test.

The same care belonged in the release itself. Keeping 2.x available meant preserving a working path for people who could not move

immediately. Making 3.x the main line meant explaining its configuration changes before somebody pulled an image. Retiring 1.x did not mean pretending it had never mattered. Its tags still held history, and its mistakes had supplied a good deal of the education.

I owe a great deal to the people who stayed with that process. QuackMasterDan, wlowen and JeslynMcKenzie kept supplying the sort of reports that let a maintainer do more than guess. Christian Gillinger gave the interface a visual identity. NewBlade made substantial improvements to the second architecture. dnordel helped take the project somewhere I had stopped being able to see from 2.x. StealthyDruid and xyxxyxxy tested through every beta of both major lines. Other names appear throughout this account because their particular observation or contribution changed what happened next.

The project has grown much bigger than I expected. I said that publicly at beta4, and publishing a stable version does not make the admission less true. There is still judgement involved in deciding what to add, what to keep, what to retire and how much change a person trying to host a film should have to absorb.

For the moment, Director's Cut is out. The old development notes are preserved, the stable release has its own account, and the announcement has its joke. I can look back through the work without having to pretend that I understood it all the first time.

The next good report may still take my brain a while to catch up with. But when it does, I want the record to be there. Wait a minute. I have seen this before.

SOURCES AND READING NOTES

The record behind the story

The full commit bodies were read, including merge and dependency-update messages, across three non-overlapping research sets of 336 records. The unique corpus is the union of current main, watchparty2 and the surviving 1.x version tags, ending at the stable 3.0.0 release commit.

Reference points: main at 5c9cd91; watchparty2 at 5318de7; v1.6.7 at f72d126. The two later branch histories share ancestors, so their individual totals are not added together. The 2.x summary retained at v2.1.0 is byte-identical to the summary on watchparty2 and was read once.

The supplied CHANGELOG-1.x.md was read alongside the recovered tag history. Both current changelogs, the preserved beta archive and the earlier 3.0 technical summary were included. This preserves early accounts while allowing later commits to correct them.

Release headings occasionally differ from GitHub publication timestamps. Broad period labels are used where that difference has no bearing on the story; publication times are interpreted in Europe/Berlin. The first public release is October 20, 2025 locally. Stable 3.0.0 was published November 2, 2026, at 07:50:38 Berlin time.

The September reflection and the release-preparation comments supply the personal voice, including the couch announcement, the value of detailed issues and the recollection triggered by break_non_keyframes. The later Ged investigation is described as an investigation, not a solved incident.

Primary records

01 The maintainer's reflection, September 12, 2026

02 Recovered 1.x history at v1.6.7

03 1.x changelog retained by its release tag

04 2.x changelog at 2.1.6

05 The eighteen-beta 2.0 development log

06 The earlier 3.0 technical development log

07 The consolidated 3.0.0 changelog

08 The preserved 3.0 beta release notes

09 Emby Quirks We Learned the Hard Way

10 Stable 3.0.0 release

11 The release announcement, November 2, 2026

Selected commit trails

01 A few friends, then a public release

fd0132e / 505c547 / 6c0d04c

02 The fix that needed another fix

3509295 / 867ef0c / 4b337cc / d099efb

03 Midnight Premiere, with additional subtitles

56c341c / 3dc96c8 / c128f9e / 9bcf819

04 Everybody gets a pause button

85167ef / 658342e / bffe8d1

05 The invisible character

51987c9 / 62370e7 / 52a654f

06 Signed, sealed, readable

8b39d49 / 4d3452f / bfe4204 / fb70758 / ec9a420

07 Director's Cut

e9ff969 / 93e007c / 9c65a22

08 Two servers, several subtitles, one film

6a45a31e / cba654d8 / a9cb16c2 / c96b93bb

09 One hundred percent of nothing

b615e941 / ec7a9f7c / ccfa22e5

10 Beta4, then the couch

f4d80df5 / 7b07f1b7 / 0a6ee088 / 5c9cd91c / 3a350ee4

Prepared after the publication of 3.0.0, November 2026. Project names and contributor handles follow the source record. The cover uses the project's existing Watch Party artwork; the timeline and stream diagrams are explanatory illustrations.